City-ζen

<<Η πόλη στα χέρια σου.>>

Project-code-v1.0

"Η έκδοση 1.0 είναι η έκδοση 0.2 του project-code με την προσθήκη του κώδικα του backend και του frontend"

ΜΕΛΗ ΟΜΑΔΑΣ

ΑΥΓΟΥΣΤΙΝΟΥ ΑΓΑΠΗ 1093327

ΔΟΥΒΡΗ ΑΓΓΕΛΙΚΗ 1097441

ΚΑΤΣΑΝΤΑΣ ΘΕΟΔΩΡΟΣ 1097459

ΜΟΝΑΣΤΗΡΙΩΤΗΣ ΗΛΙΑΣ 1093436

ΣΚΟΥΤΑΣ ΚΩΝΣΤΑΝΤΙΝΟΣ 1093498

ΣΥΝΤΑΚΤΕΣ Project-code-v1.0

Editors

ΑΥΓΟΥΣΤΙΝΟΥ ΑΓΑΠΗ 1093327

ΜΟΝΑΣΤΗΡΙΩΤΗΣ ΗΛΙΑΣ 1093436

Βάση δεδομένων

Πίνακες:

Διαχείρηση Χρηστών:

user: βασικός πίνακας για credentials/πληροφορίες σύνδεσης (username, password_hash, email, τηλέφωνο).

```
-- table for users
CREATE TABLE user (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    telephone_number VARCHAR(20) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

citizen και municipality: εξειδικεύουν τον user σε δύο ρόλους: πολίτης και δήμος, με ξένο κλειδί στο user (το user_id).

```
-- table for municipality
CREATE TABLE municipality (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    municipality VARCHAR(100) NOT NULL,
    contact_email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE
);
```

```
-- table for citizens
CREATE TABLE citizen (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    municipality_id INT NOT NULL,
    name VARCHAR(100),
    points INT DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES user(id) ON DELETE CASCADE,
    FOREIGN KEY (municipality_id) REFERENCES municipality(id) ON DELETE CASCADE
);
```

Διαχείρηση Αναφορών:

report: ο κύριος πίνακας αναφοράς, με χρήσιμα πεδία (status, priority, category, etc.).

location: σωστά διαχωρισμένος για επαναχρησιμοποίηση τοποθεσιών.

report_media: media που σχετίζονται με αναφορές.

```
-- table for reports
CREATE TABLE report (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    location_id INT NOT NULL,
    category ENUM('Lighting', 'Roadworks', 'Other') NOT NULL DEFAULT 'Other',
    description TEXT NOT NULL,
    priority ENUM('High', 'Medium', 'Low') NOT NULL DEFAULT 'Medium',
    status ENUM('Pending', 'Rejected', 'In Progress', 'Completed') NOT NULL DEFAULT 'Pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (location_id) REFERENCES location(id)
    );

-- table for location
CREATE TABLE location (
    id INT AUTO_INCREMENT PRIMARY KEY,
    latitude DOUBLE NOT NULL,
    longitude DOUBLE NOT NULL,
    address VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- table for report media
CREATE TABLE report_media (
    id INT AUTO_INCREMENT PRIMARY KEY,
    report_id INT NOT NULL,
    media_url VARCHAR(500) NOT NULL,
    media_type ENUM('image', 'video') DEFAULT 'image',
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (report_id) REFERENCES report(id) ON DELETE CASCADE
);
```

Σχόλια και Επικοινωνία:

citizen_comment: σωστή δομή με parent_comment_id για υποστήριξη απαντήσεων (nested threads).

municipality_comment: ξεχωριστός πίνακας για σχόλια από τον δήμο — ενισχύει τη διαφάνεια.

```
-- table for citizen comments
```

```
CREATE TABLE citizen_comment (
    id INT AUTO_INCREMENT PRIMARY KEY,
    report_id INT NOT NULL,
    citizen_id INT NOT NULL,
    parent_comment_id INT NULL,
    comment TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (report_id) REFERENCES report(id),
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (parent_comment_id) REFERENCES citizen_comment(id) ON DELETE CASCADE
);

-- table for municipality comments
CREATE TABLE municipality_comment (
    id INT AUTO_INCREMENT PRIMARY KEY,
    municipality_id INT NOT NULL,
    report_id INT NOT NULL,
    comment TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (municipality_id) REFERENCES municipality(id) ON DELETE CASCADE,
    FOREIGN KEY (report_id) REFERENCES report(id) ON DELETE CASCADE
);
```

AI & Bot Συνομιλίες:

ai_conversation / ai_chat: καλός διαχωρισμός συνομιλίας και επιμέρους μηνυμάτων.

Περιλαμβάνει χρήσιμες πληροφορίες (π.χ. action_type, report_id) — έτοιμο για analytics/κατηγοριοποίηση.

```
-- table for ai conversation
CREATE TABLE ai_conversation (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    started_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ended_at TIMESTAMP,
    resolved BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id)
);

-- table for ai chat
CREATE TABLE ai_chat (
    id INT AUTO_INCREMENT PRIMARY KEY,
    conversation_id INT NOT NULL,
    citizen_id INT NOT NULL,
    message TEXT NOT NULL,
    is_user BOOLEAN NOT NULL,
    report_id INT,
    action_type ENUM('submit_report', 'check_status', 'get_info', 'feedback', 'other'),
    feedback_rating TINYINT,
    user_feedback TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (conversation_id) REFERENCES ai_conversation(id) ON DELETE
CASCADE,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (report_id) REFERENCES report(id)
);
```

Ειδοποιήσεις & Ανακοινώσεις:

notification: πλήρης πίνακας με αναφορά στο action_type, citizen_id, report_id.

municipality_post: ανακοινώσεις από δήμο, με δυνατότητα εικόνας/media.

```
-- table for notifications
CREATE TABLE notification (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    report_id INT DEFAULT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action_type ENUM(
        'report_submitted',
        'status_updated',
        'points_awarded',
        'points_deducted',
        'admin_announcement',
        'general'
    ) NOT NULL,
    link_url VARCHAR(255),
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (report_id) REFERENCES report(id)
);

-- table for municipality announcements
CREATE TABLE municipality_post (
    id INT AUTO_INCREMENT PRIMARY KEY,
    municipality_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
```

```
    text TEXT NOT NULL,
    media_url VARCHAR(500),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (municipality_id) REFERENCES municipality(id)
);
```

Πόντοι, Επιβραβεύσεις και Κυρώσεις:

points / points_redemption / false_report / penalty: ολοκληρωμένο point system με tracking αιτίας (reason), επιβραβεύσεις και ποινές.

```
-- table for points
CREATE TABLE points (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    report_id INT DEFAULT NULL,
    points_awarded INT NOT NULL,
    reason VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (report_id) REFERENCES report(id)
);

-- table for point redemptions
CREATE TABLE points_redemption (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    points_spent INT NOT NULL,
    reward_description VARCHAR(255) NOT NULL,
    redeemed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id)
```

```
);

-- table for false reports
CREATE TABLE false_report (
    id INT AUTO_INCREMENT PRIMARY KEY,
    report_id INT NOT NULL UNIQUE,
    detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    reason TEXT,
    FOREIGN KEY (report_id) REFERENCES report(id) ON DELETE CASCADE
);


-- table for penalties
CREATE TABLE penalty (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    false_report_id INT NOT NULL,
    points_deducted INT NOT NULL,
    reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
    FOREIGN KEY (false_report_id) REFERENCES report(id)
);
```

Αξιολόγηση Αναφορών:

report_rating: Δυνατότητα αξιολόγησης επίλυσης από τον πολίτη με επιμέρους βαθμολογίες (χρόνος, επικοινωνία, ποιότητα). Ενισχύει τη διαφάνεια και ανατροφοδότηση προς τον δήμο.

```
-- table for report rating
```

```
CREATE TABLE report_rating (
    id INT AUTO_INCREMENT PRIMARY KEY,
    report_id INT NOT NULL UNIQUE,
    citizen_id INT NOT NULL,
    resolution_time_rating TINYINT NOT NULL CHECK (resolution_time_rating BETWEEN 1 AND 5),
    communication_rating TINYINT NOT NULL CHECK (communication_rating BETWEEN 1 AND 5),
    solution_quality_rating TINYINT NOT NULL CHECK (solution_quality_rating BETWEEN 1 AND 5),
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (report_id) REFERENCES report(id),
    FOREIGN KEY (citizen_id) REFERENCES citizen(id)
);
```

Αιτήματα Πολιτών:

request: Καταγράφει αιτήματα πολιτών προς τον δήμο με κατάσταση επεξεργασίας (Pending, Approved, Rejected).

```
-- table for requests
CREATE TABLE request (
    id INT AUTO_INCREMENT PRIMARY KEY,
    citizen_id INT NOT NULL,
    municipality_id INT NOT NULL,
    description TEXT NOT NULL,
    status ENUM('Pending', 'Approved', 'Rejected') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (citizen_id) REFERENCES citizen(id),
```

```
    FOREIGN KEY (municipality_id) REFERENCES municipality(id)
);
```

Procedures:

SubmitReport: Εισάγει νέα αναφορά από πολίτη με τις λεπτομέρειες της και δημιουργεί σχετική ειδοποίηση στον πολίτη.

UpdateReportStatus: Ενημερώνει την κατάσταση μιας αναφοράς και απονέμει ή αφαιρεί πόντους στον πολίτη, με ειδοποίηση για την αλλαγή κατάστασης.

AwardPoints: Απονέμει πόντους στον πολίτη για την ολοκλήρωση μιας αναφοράς και στέλνει ειδοποίηση.

DeductPoints: Αφαιρεί πόντους από τον πολίτη για λάθος αναφορά και αποστέλλει ειδοποίηση για την ποινή.

SubmitRequest: Υποβάλλει αίτημα από πολίτη στον δήμο και δημιουργεί σχετική ειδοποίηση για την υποβολή του.

PublishMunicipalityPost: Δημιουργεί ανάρτηση από τον δήμο και ενημερώνει όλους τους πολίτες.

GetOtherCitizensInProgressReports: Εμφανίζει αναφορές σε εκκρεμότητα από άλλους πολίτες στο ίδιο δήμο.

AddCitizenComment: Επιτρέπει σε πολίτη να σχολιάσει μια αναφορά άλλου πολίτη και ενημερώνει τον κάτοχο της αναφοράς.

SubmitFeedback: Υποβάλλει ανατροφοδότηση από τον πολίτη μέσω της AI συνομιλίας και καταγράφει τη βαθμολογία και το σχόλιο.

CreateCitizen: Δημιουργεί λογαριασμό πολίτη με τα προσωπικά του στοιχεία.

CreateMunicipality: Δημιουργεί λογαριασμό δήμου με τα στοιχεία επαφής και σύνδεσης.

GetCitizenReports: Εμφανίζει τις αναφορές που έχει υποβάλει ο συγκεκριμένος πολίτης.

GetReportsForMunicipality: Εμφανίζει τις αναφορές των πολιτών για ένα συγκεκριμένο δήμο.

AddMunicipalityComment: Επιτρέπει στον δήμο να σχολιάσει μια αναφορά και ειδοποιεί τον πολίτη.

RedeemPoints: Επιτρέπει στον πολίτη να εξαργυρώσει πόντους για ανταμοιβή και καταγράφει την εξόφληση.

GetCitizenPoints: Εμφανίζει τους πόντους που διαθέτει ο πολίτης.

GetCitizenNotifications: Εμφανίζει όλες τις ειδοποιήσεις για έναν πολίτη.

EndAIConversation: Τερματίζει τη συνομιλία AI αν το πρόβλημα έχει λυθεί.

auto_close_inactive_conversations: Κλείνει αυτόματα τις ανενεργές συνομιλίες AI μετά από 1 ώρα αδράνειας.

EditCitizenFullProfileWithPassword: Επιτρέπει στον πολίτη να επεξεργαστεί το προφίλ του, περιλαμβανομένων των στοιχείων σύνδεσης.

ReplyToCitizenComment: Επιτρέπει σε πολίτη να απαντήσει σε σχόλιο άλλου πολίτη σε αναφορά και ενημερώνει τον αρχικό σχολιαστή.

```
-- citizen submits report and gets notification

DELIMITER //


CREATE PROCEDURE SubmitReport (
    IN p_citizen_id INT,
    IN p_latitude DOUBLE,
    IN p_longitude DOUBLE,
    IN p_address VARCHAR(255),
    IN p_category ENUM('lighting', 'roadworks', 'other'),
    IN p_description TEXT,
    IN p_priority ENUM('High', 'Medium', 'Low'),
    IN p_media_url VARCHAR(500),
    IN p_media_type ENUM('image', 'video')
)
BEGIN
    DECLARE loc_id INT;
    DECLARE report_id INT;


    INSERT INTO location (latitude, longitude, address)
    VALUES (p_latitude, p_longitude, p_address);
```

```
    SET loc_id = LAST_INSERT_ID();

    INSERT INTO report (citizen_id, location_id, category, description, priority)
    VALUES (p_citizen_id, loc_id, p_category, p_description, p_priority);
    SET report_id = LAST_INSERT_ID();

    IF p_media_url IS NOT NULL AND p_media_url != '' THEN
        INSERT INTO report_media (report_id, media_url, media_type)
        VALUES (report_id, p_media_url, p_media_type);
    END IF;

    INSERT INTO notification (citizen_id, message, action_type, report_id)
    VALUES (
        p_citizen_id,
        CONCAT('Your report has been submitted successfully. Report ID: ', report_id),
        'report_submitted',
        report_id
    );
END //

DELIMITER ;




-- municipality changes status, citizen gets notified and if new status is 'in progress' gets awarded points, if it is 'rejected' gets deducted points
DELIMITER //
```

```
CREATE PROCEDURE UpdateReportStatus (
    IN p_report_id INT,
    IN p_citizen_id INT,
    IN p_new_status ENUM('Pending', 'Rejected', 'In Progress', 'Completed')
)
BEGIN
    UPDATE report
    SET status = p_new_status
    WHERE id = p_report_id;

    IF p_new_status = 'In progress' THEN
        CALL AwardPoints(p_citizen_id, p_report_id, 10, 'Report completed');
    END IF;

    IF p_new_status = 'Rejected' THEN
        CALL DeductPoints(p_citizen_id, p_report_id, 10, 'Report was rejected');
    END IF;

    INSERT INTO notification (citizen_id, message, action_type, report_id)
    VALUES (
        p_citizen_id,
        CONCAT('Status of your report (ID: ', p_report_id, ') has been updated to ',
p_new_status),
        'status_updated',
        p_report_id
    );
END //
```

```
DELIMITER ;




-- citizen gets awarded points and gets notification
DELIMITER //

CREATE PROCEDURE AwardPoints (
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_points INT,
    IN p_reason VARCHAR(255)
)
BEGIN
    INSERT INTO points (citizen_id, report_id, points_awarded, reason)
    VALUES (p_citizen_id, p_report_id, p_points, p_reason);

    UPDATE citizen
    SET points = points + p_points
    WHERE id = p_citizen_id;

    INSERT INTO notification (citizen_id, message, action_type, report_id)
    VALUES (
        p_citizen_id,
        CONCAT('You have been awarded ', p_points, ' points: ', p_reason),
        'points_awarded',
        p_report_id
```

```
    );
END //

DELIMITER ;



-- citizen gets deducted points and gets notification
DELIMITER //

CREATE PROCEDURE DeductPoints(
    IN p_citizen_id INT,
    IN p_false_report_id INT,
    IN p_points_deducted INT,
    IN p_reason TEXT
)
BEGIN
    INSERT INTO penalty (citizen_id, false_report_id, points_deducted, reason)
    VALUES (p_citizen_id, p_false_report_id, p_points_deducted, p_reason);

    UPDATE citizen
    SET points = points - p_points_deducted
    WHERE id = p_citizen_id;

    INSERT INTO notification (citizen_id, message, action_type, report_id)
    VALUES (
        p_citizen_id,
        CONCAT('Warning: ', p_points_deducted, ' points deducted. Reason: ', p_reason),
```

```
        'points_deducted',
        p_false_report_id
    );
END //

DELIMITER ;




-- citizen submits a request
DELIMITER //

CREATE PROCEDURE SubmitRequest (
    IN p_citizen_id INT,
    IN p_municipality_id INT,
    IN p_description TEXT
)
BEGIN
    DECLARE new_request_id INT;

    INSERT INTO request (citizen_id, municipality_id, description, status)
    VALUES (p_citizen_id, p_municipality_id, p_description, 'Pending');
    SET new_request_id = LAST_INSERT_ID();

    INSERT INTO notification (citizen_id, message, action_type, link_url)
    VALUES (
        p_citizen_id,
        CONCAT('Your request has been submitted successfully. Request ID: ',
new_request_id),
```

```
        'general',
        CONCAT('/requests/', new_request_id)
    );
END //

DELIMITER ;




DELIMITER //

CREATE PROCEDURE PublishMunicipalityPost (
    IN p_municipality_id INT,
    IN p_title VARCHAR(255),
    IN p_content TEXT,
    IN p_media_url VARCHAR(500)
)
BEGIN
    DECLARE post_id INT;

    INSERT INTO municipality_post (municipality_id, title, text, media_url)
    VALUES (p_municipality_id, p_title, p_content, p_media_url);
    SET post_id = LAST_INSERT_ID();

    INSERT INTO notification (citizen_id, message, action_type)
    SELECT id, CONCAT('New municipality announcement: ', p_title),
'admin_announcement'
    FROM citizen;
END //
```

```
DELIMITER ;




-- citizen views other citizens' reports that are in progress
DELIMITER //

CREATE PROCEDURE GetOtherCitizensInProgressReports (
    IN p_citizen_id INT
)
BEGIN
    DECLARE v_municipality_id INT;

    SELECT municipality_id INTO v_municipality_id
    FROM citizen
    WHERE id = p_citizen_id;

    SELECT
        r.id AS report_id,
        r.description,
        r.status,
        r.created_at,
        l.address AS location_address,
        l.latitude,
        l.longitude,
        c.id AS reporter_id,
```

```
        c.name AS reporter_name,
        GROUP_CONCAT(DISTINCT rm.media_url SEPARATOR ', ') AS media_urls,
        GROUP_CONCAT(DISTINCT CONCAT(cc.comment, ' (by Citizen ID: ', cc.citizen_id,
')') SEPARATOR ' || ') AS comments
    FROM report r
    JOIN citizen c ON r.citizen_id = c.id
    JOIN location l ON r.location_id = l.id
    LEFT JOIN report_media rm ON r.id = rm.report_id
    LEFT JOIN citizen_comment cc ON r.id = cc.report_id
    WHERE r.status = 'In Progress'
      AND c.municipality_id = v_municipality_id
      AND c.id != p_citizen_id
    GROUP BY r.id
    ORDER BY r.created_at ASC;
END //

DELIMITER ;




-- citizen comments on another citizen's report and citizen gets notification
DELIMITER //

CREATE PROCEDURE AddCitizenComment (
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_comment TEXT
)
BEGIN
```

```
    DECLARE report_owner_id INT;

    INSERT INTO citizen_comment (report_id, citizen_id, comment)
    VALUES (p_report_id, p_citizen_id, p_comment);

    SELECT citizen_id INTO report_owner_id
    FROM report
    WHERE id = p_report_id;

    IF report_owner_id IS NOT NULL AND report_owner_id != p_citizen_id THEN
        INSERT INTO notification (citizen_id, message, action_type, report_id)
        VALUES (
            report_owner_id,
            CONCAT('Another citizen commented on your report (ID: ', p_report_id, ').'),
            'general',
            p_report_id
        );
    END IF;
END //

DELIMITER ;




-- citizen submits feedback in ai chat
DELIMITER //

CREATE PROCEDURE SubmitFeedback (
```

```
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_rating TINYINT,
    IN p_comment TEXT
)
BEGIN
    DECLARE conv_id INT;

    INSERT INTO ai_conversation (citizen_id)
    VALUES (p_citizen_id);
    SET conv_id = LAST_INSERT_ID();

    INSERT INTO ai_chat (conversation_id, citizen_id, message, is_user, report_id,
action_type, feedback_rating, user_feedback)
    VALUES (conv_id, p_citizen_id, p_comment, TRUE, p_report_id, 'feedback', p_rating,
p_comment);
END //

DELIMITER ;




-- citizen creates account
DELIMITER //

CREATE PROCEDURE CreateCitizen(
    IN p_username VARCHAR(100),
    IN p_password_hash VARCHAR(255),
    IN p_email VARCHAR(100),
```

```
    IN p_telephone_number VARCHAR(20),
    IN p_municipality_id INT
)
BEGIN
    INSERT INTO user (username, password_hash, email, telephone_number)
    VALUES (p_username, p_password_hash, p_email, p_telephone_number);

    INSERT INTO citizen (user_id, municipality_id)
    VALUES (LAST_INSERT_ID(), p_municipality_id);
END //

DELIMITER ;



-- municipality creates account
DELIMITER //

CREATE PROCEDURE CreateMunicipality(
    IN p_username VARCHAR(100),
    IN p_password_hash VARCHAR(255),
    IN p_email VARCHAR(100),
    IN p_telephone_number VARCHAR(20),
    IN p_municipality VARCHAR(100),
    IN p_contact_email VARCHAR(100)
)
BEGIN
    INSERT INTO user (username, password_hash, email, telephone_number)
```

```
    VALUES (p_username, p_password_hash, p_email, p_telephone_number);

    INSERT INTO municipality (user_id, municipality, contact_email)
    VALUES (LAST_INSERT_ID(), p_municipality, p_contact_email);
END //

DELIMITER ;



-- citizen views their own reports
DELIMITER //

CREATE PROCEDURE GetCitizenReports(
    IN p_citizen_id INT
)
BEGIN
    SELECT
        r.id AS report_id,
        r.description,
        r.category,
        r.priority,
        r.status,
        r.created_at,
        l.address AS location_address,
        l.latitude,
        l.longitude,
        (
```

```
        SELECT GROUP_CONCAT(rm.media_url SEPARATOR ', ')
        FROM report_media rm
        WHERE rm.report_id = r.id
    ) AS media_urls,
    (
        SELECT GROUP_CONCAT(CONCAT(cit.name, ': ', cc.comment) SEPARATOR ' || ')
        FROM citizen_comment cc
        JOIN citizen cit ON cit.id = cc.citizen_id
        WHERE cc.report_id = r.id
    ) AS comments
  FROM report r
  JOIN location l ON r.location_id = l.id
  WHERE r.citizen_id = p_citizen_id
  ORDER BY r.created_at DESC;
END //

DELIMITER ;




-- municipality views citizens' reports
DELIMITER //

CREATE PROCEDURE GetReportsForMunicipality(
  IN p_municipality_id INT
)
BEGIN
  SELECT
```

```
        r.id AS report_id,
        r.description,
        r.category,
        r.priority,
        r.status,
        r.created_at,
        l.address AS location_address,
        l.latitude,
        l.longitude,
        (
            SELECT GROUP_CONCAT(rm.media_url SEPARATOR ', ')
            FROM report_media rm
            WHERE rm.report_id = r.id
        ) AS media_urls,
        (
            SELECT GROUP_CONCAT(CONCAT(cit.name, ': ', cc.comment) SEPARATOR ' || ')
            FROM citizen_comment cc
            JOIN citizen cit ON cit.id = cc.citizen_id
            WHERE cc.report_id = r.id
        ) AS comments
    FROM report r
    INNER JOIN location l ON r.location_id = l.id
    INNER JOIN citizen c ON r.citizen_id = c.id
    WHERE c.municipality_id = p_municipality_id
    ORDER BY r.created_at DESC;
END //

DELIMITER ;
```

```
-- municipality comments on a report and citizen gets notified
DELIMITER //

CREATE PROCEDURE AddMunicipalityComment(
    IN p_municipality_id INT,
    IN p_report_id INT,
    IN p_comment TEXT
)
BEGIN
    INSERT INTO municipality_comment (municipality_id, report_id, comment)
    VALUES (p_municipality_id, p_report_id, p_comment);

    INSERT INTO notification (citizen_id, report_id, message, action_type)
    SELECT r.citizen_id, r.id, CONCAT('Υπάρχει νέο σχόλιο από τον Δήμο στη αναφορά σας.'), 'general'
    FROM report r
    WHERE r.id = p_report_id;

END //

DELIMITER ;




-- citizen redeems points
DELIMITER //
```

```
CREATE PROCEDURE RedeemPoints(
    IN p_citizen_id INT,
    IN p_points_spent INT,
    IN p_reward_description VARCHAR(255)
)
BEGIN
    DECLARE current_points INT;

    SELECT points INTO current_points
    FROM citizen
    WHERE id = p_citizen_id;

    IF current_points >= p_points_spent THEN

        UPDATE citizen
        SET points = points - p_points_spent
        WHERE id = p_citizen_id;

        INSERT INTO points_redemption (citizen_id, points_spent, reward_description)
        VALUES (p_citizen_id, p_points_spent, p_reward_description);
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Not enough points';
    END IF;
END //

DELIMITER ;
```

```
-- citizen views how many points they have
DELIMITER //

CREATE PROCEDURE GetCitizenPoints(
    IN p_citizen_id INT
)
BEGIN
    SELECT points
    FROM citizen
    WHERE id = p_citizen_id;
END //

DELIMITER ;




-- citizen views all notifications
DELIMITER //




-- citizen views all notifications
CREATE PROCEDURE GetCitizenNotifications(
    IN p_citizen_id INT
)
BEGIN
```

```
    SELECT * FROM notification
    WHERE citizen_id = p_citizen_id
    ORDER BY created_at DESC;
END //

DELIMITER ;



-- ai conversation is ended if the issue is resolved
DELIMITER //

CREATE PROCEDURE EndAIConversation (
    IN p_conversation_id INT
)
BEGIN
    UPDATE ai_conversation
    SET ended_at = CURRENT_TIMESTAMP, resolved = TRUE
    WHERE id = p_conversation_id;
END //

DELIMITER ;



-- ai conversation is ended if user is inactive for more than one hour
DELIMITER //
```

```
CREATE EVENT auto_close_inactive_conversations
ON SCHEDULE EVERY 5 MINUTE
DO
BEGIN
   UPDATE ai_conversation ac
   SET ac.ended_at = CURRENT_TIMESTAMP, ac.resolved = FALSE
   WHERE ac.ended_at IS NULL
     AND (
        SELECT MAX(chat.created_at)
        FROM ai_chat chat
        WHERE chat.conversation_id = ac.id
     ) < NOW() - INTERVAL 1 HOUR;
END //

DELIMITER ;




-- citizen edits profile
DELIMITER //

CREATE PROCEDURE EditCitizenFullProfileWithPassword (
   IN p_citizen_id INT,
   IN p_new_name VARCHAR(100),
   IN p_new_municipality_id INT,
   IN p_new_username VARCHAR(100),
   IN p_new_email VARCHAR(100),
   IN p_new_telephone_number VARCHAR(20),
```

```
    IN p_new_password_hash VARCHAR(255)
)
BEGIN
    DECLARE v_user_id INT;

    SELECT user_id INTO v_user_id
    FROM citizen
    WHERE id = p_citizen_id;

    UPDATE citizen
    SET
        name = p_new_name,
        municipality_id = p_new_municipality_id
    WHERE id = p_citizen_id;

    UPDATE user
    SET
        username = p_new_username,
        email = p_new_email,
        telephone_number = p_new_telephone_number,
        password_hash = p_new_password_hash
    WHERE id = v_user_id;
END //

DELIMITER ;
```

```
-- citizen replies to a citizen that has commented under their report
DELIMITER //

CREATE PROCEDURE ReplyToCitizenComment (
    IN p_citizen_id INT,
    IN p_report_id INT,
    IN p_parent_comment_id INT,
    IN p_reply_text TEXT
)
BEGIN
    DECLARE original_commenter_id INT;

    INSERT INTO citizen_comment (report_id, citizen_id, comment,
parent_comment_id)
    VALUES (p_report_id, p_citizen_id, p_reply_text, p_parent_comment_id);

    SELECT citizen_id INTO original_commenter_id
    FROM citizen_comment
    WHERE id = p_parent_comment_id;

    IF original_commenter_id IS NOT NULL AND original_commenter_id != p_citizen_id
THEN
        INSERT INTO notification (citizen_id, message, action_type, report_id)
        VALUES (
            original_commenter_id,
            CONCAT('A citizen replied to your comment on report ID ', p_report_id),
            'general',
            p_report_id
        );
```

```
    END IF;
END //

DELIMITER
```

BACKEND

Controllers:

Οι controllers αποτελούν βασικό μέρος της αρχιτεκτονικής μιας web εφαρμογής. Είναι υπεύθυνοι για την επεξεργασία των αιτημάτων (requests) που λαμβάνονται από τον client (π.χ. frontend) και την επιστροφή των κατάλληλων απαντήσεων (responses).

Πιο συγκεκριμένα, οι controllers:

- Διαβάζουν δεδομένα από το αίτημα.

- Εκτελούν την επιχειρησιακή λογική (business logic).

- Αλληλεπιδρούν με τη βάση δεδομένων.

- Επιστρέφουν τα αποτελέσματα ή τυχόν σφάλματα στον client.

Ο διαχωρισμός της λογικής σε controllers προσφέρει οργανωμένη δομή, ευκολία συντήρησης, και καλύτερη επεκτασιμότητα της εφαρμογής.

Για τις ανάγκες της δικής μας εφαρμογής δημιουργήσαμε τους εξής controllers:

1) authController.js

```
// controllers/authController.js
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const pool = require('../db');

const JWT_SECRET = process.env.JWT_SECRET || 'your_jwt_secret_here';
```

```
// POST /login
exports.login = async (req, res) => {
  const { username, password } = req.body;
  console.log(`Login attempt for username: ${username}`);

  try {
    const [userRows] = await pool.execute('SELECT * FROM user WHERE username =
?', [username]);

    if (userRows.length === 0) {
      console.log(`User not found: ${username}`);
      return res.status(401).json({ message: 'Invalid credentials' });
    }

    const user = userRows[0];
    const isMatch = await bcrypt.compare(password, user.password_hash);

    if (!isMatch) {
      console.log(`Password mismatch for user: ${username}`);
      return res.status(401).json({ message: 'Invalid credentials' });
    }

    // 🔍 Get matching citizen ID (assuming 1-to-1 user ↔ citizen)
    const [citizenRows] = await pool.execute(
      'SELECT id FROM citizen WHERE user_id = ?',
      [user.id]
    );
```

```
    const citizenId = citizenRows.length > 0 ? citizenRows[0].id : null;

    // 🛡️ Include citizenId in JWT
    const token = jwt.sign(
      { id: user.id, username: user.username, citizenId },
      JWT_SECRET,
      { expiresIn: '1h' }
    );

    console.log(`Login successful for user: ${username}`);
    res.json({ token });
  } catch (error) {
    console.error('Login error:', error);
    res.status(500).json({ message: 'Server error' });
  }
};

// GET /users/me
exports.getCurrentUser = async (req, res) => {
  try {
    const [rows] = await pool.execute(
      'SELECT id, username, email FROM user WHERE id = ?',
      [req.user.id]
    );

    if (rows.length === 0) {
      return res.status(404).json({ message: 'User not found' });
    }
```

```
    res.json(rows[0]);
  } catch (error) {
    console.error('Error fetching user info:', error);
    res.status(500).json({ message: 'Server error' });
  }
};
```

2) pointsController.js

```
const pool = require('../db');

exports.getMyPoints = async (req, res) => {
  try {
    const citizenId = req.user.citizenId;
    if (!citizenId) {
      return res.status(400).json({ error: 'Citizen ID not found in token' });
    }

    // Call stored procedure to get points
    const [rows] = await pool.query('CALL GetCitizenPoints(?)', [citizenId]);

    // The result of CALL is an array: rows[0] is the actual result set
    const pointsRow = rows[0][0];

    // Default to 0 if no points found
    const totalPoints = pointsRow ? pointsRow.points : 0;
```

```
    res.json({ points: totalPoints });
  } catch (error) {
    console.error('Error fetching points:', error);
    res.status(500).json({ error: 'Server error while fetching points' });
  }
};
```

3) reportController.js

```
// controllers/reportController.js
const pool = require('../db');
const multer = require('multer');
const path = require('path');

// Multer setup (for image upload)
const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'uploads/');
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + path.extname(file.originalname));
  },
});
const upload = multer({ storage: storage });
```

```
// Exported multer upload middleware
exports.upload = upload.single('image');

// Controller: GET all reports
exports.getAllReports = async (req, res) => {
  try {
    const [rows] = await pool.query(`
      SELECT r.id, r.category, r.description, r.priority, r.status, r.created_at,
             c.name AS citizen_name,
             l.address AS location_address
      FROM report r
      JOIN citizen c ON r.citizen_id = c.id
      JOIN location l ON r.location_id = l.id
      ORDER BY r.created_at DESC
    `);

    res.json(rows);
  } catch (error) {
    console.error('Error fetching reports:', error);
    res.status(500).json({ message: 'Server error' });
  }
};

// Controller: Create a new report
exports.createReport = async (req, res) => {
  try {
    const { latitude, longitude, address, category, comment, citizen_id } = req.body;
    const imageFile = req.file;
```

```
    if (!latitude || !longitude || !address || !category || !imageFile || !citizen_id) {
      return res.status(400).json({ message: 'Missing required fields' });
    }

    const [locationResult] = await pool.query(
      'INSERT INTO location (latitude, longitude, address) VALUES (?, ?, ?)',
      [latitude, longitude, address]
    );

    const location_id = locationResult.insertId;

    const [reportResult] = await pool.query(
      `INSERT INTO report (citizen_id, location_id, category, description, status,
created_at, image_path)
       VALUES (?, ?, ?, ?, ?, NOW(), ?)`,
      [citizen_id, location_id, category, comment || '', 'pending', imageFile.filename]
    );

    res.json({ message: 'Report created successfully', reportId: reportResult.insertId
});
  } catch (error) {
    console.error('Error creating report:', error);
    res.status(500).json({ message: 'Server error' });
  }
};
```

4) newsController.js

```
onst db = require('../db'); // ή η σύνδεσή σου με τη βάση

const getAllMunicipalityPosts = async (req, res) => {
  try {
    const [rows] = await db.query(`
      SELECT
        mp.id, mp.title, mp.text, mp.media_url, mp.created_at,
        m.name AS municipality_name
      FROM municipality_post mp
      JOIN municipality m ON mp.municipality_id = m.id
      ORDER BY mp.created_at DESC
    `);

    res.json(rows);
  } catch (err) {
    console.error('❌ Error fetching municipality posts:', err);
    res.status(500).json({ error: 'Internal Server Error' });
  }
};

module.exports = { getAllMunicipalityPosts };
```

Middleware:

Τα middleware είναι συναρτήσεις που εκτελούνται κατά τη διάρκεια της επεξεργασίας ενός αιτήματος (request) στον διακομιστή (server), πριν φτάσει στον τελικό controller ή πριν σταλεί η απάντηση (response).

Συνήθως χρησιμοποιούνται για:

- Έλεγχο ταυτότητας (authentication) και εξουσιοδότησης (authorization).

- Καταγραφή των αιτημάτων (logging).

- Επεξεργασία δεδομένων, όπως το parsing του req.body.

- Διαχείριση σφαλμάτων.

Ένα middleware μπορεί είτε να συνεχίσει τη ροή (με next()), είτε να διακόψει την εκτέλεση επιστρέφοντας απάντηση.

Ο ρόλος τους είναι να προσθέτουν επιπλέον λειτουργίες σε ένα request/response χωρίς να μπλέκεται ο βασικός controller.

Εμείς βάλαμε τις εξής:

1) authMiddleware.js

```
// middleware/authMiddleware.js
const jwt = require('jsonwebtoken');

function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.sendStatus(401);

  console.log('JWT_SECRET in middleware:', process.env.JWT_SECRET);

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user; // Add user info to request
    next();
```

```
  });
}

module.exports = authenticateToken;
```

Routes:

Τα routes (δρομολογήσεις) καθορίζουν ποιο κομμάτι του backend κώδικα θα εκτελεστεί όταν το backend λάβει ένα συγκεκριμένο HTTP αίτημα (π.χ. GET, POST) σε μια συγκεκριμένη διαδρομή (π.χ. /api/reports).

Κάθε route αντιστοιχεί σε μία URL διαδρομή και καλεί τον αντίστοιχο controller, ο οποίος περιέχει τη λογική του συστήματος.

Παραδείγματα χρήσης routes:

- Για να λάβουμε δεδομένα από τη βάση (GET).

- Για να καταχωρίσουμε μια νέα εγγραφή (POST).

- Για να ενημερώσουμε ή να διαγράψουμε δεδομένα (PUT/DELETE).

Ουσιαστικά, τα routes είναι το σημείο εισόδου για κάθε λειτουργία της εφαρμογής, και οργανώνουν τον κώδικα ώστε να είναι επεκτάσιμος και καθαρός.

Στο πρότζεκτ μας έχουμε φτιάξει τα εξής:

1) authRoutes.js

```
// routes/authRoutes.js

const express = require('express');

const router = express.Router();

const authController = require('../controllers/authController');

const authenticateToken = require('../middleware/authMiddleware');
```

```
// POST login
router.post('/login', authController.login);

// GET current user
router.get('/users/me', authenticateToken, authController.getCurrentUser);

module.exports = router;
```

2) pointsRoutes.js

```
const express = require('express');
const router = express.Router();
const authenticateToken = require('../middleware/authMiddleware');
const { getMyPoints } = require('../controllers/pointsController');

router.get('/me', authenticateToken, getMyPoints);

module.exports = router;  // Make sure this is here!
```

3) reportRoutes.js

```
// routes/reportRoutes.js
const express = require('express');
const router = express.Router();
const reportController = require('../controllers/reportController');
```

```
// GET all reports
router.get('/', reportController.getAllReports);

// POST create report with image upload
router.post('/create', reportController.upload, reportController.createReport);

module.exports = router;
```

4) newsRoutes.js

```
const express = require('express');
const router = express.Router();
const { getAllMunicipalityPosts } = require('../controllers/newsController');

// Use the controller directly
router.get('/posts', getAllMunicipalityPosts);

module.exports = router;
```

5) userreportsRoutes

```
console.log(' 🔹 userreportsRoutes loaded');

const express = require('express');
const router = express.Router();
const db = require('../db');  // Σιγουρέψου ότι το path προς το db είναι σωστό
```

```
// GET reports for a specific citizen
router.get('/citizen/:id', async (req, res) => {
    console.log(`🔹 GET /citizen/${req.params.id} called`);

  const citizenId = req.params.id;
  try {
    const [rows] = await db.query('CALL GetCitizenReports(?)', [citizenId]);
    res.json(rows[0]); // Stored procedure returns a nested array
  } catch (err) {
    console.error('❌ Error fetching citizen reports:', err);
    res.status(500).json({ error: 'Internal server error' });
  }
});

module.exports = router;
```

.env αρχείο:

Το αρχείο .env χρησιμοποιείται για να αποθηκεύει περιβαλλοντικές μεταβλητές (environment variables) που είναι απαραίτητες για τη λειτουργία της εφαρμογής μας, χωρίς να χρειαστεί να της γράψουμε σκληρά (hardcoded) μέσα στον κώδικα.

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=root
```

```
DB_NAME=Cityzen
JWT_SECRET=your_jwt_secret_here
PORT=5000
```

db.js αρχείο:

Το αρχείο db.js είναι υπεύθυνο για τη σύνδεση του backend με τη βάση δεδομένων MySQL. Δημιουργεί και εξάγει μια πισίνα συνδέσεων (connection pool) που χρησιμοποιείται σε όλη την εφαρμογή για την εκτέλεση SQL queries.

```
// backend/db.js

require('dotenv').config(); // Load .env variables

const mysql = require('mysql2/promise');

const db = mysql.createPool({
  host: process.env.DB_HOST || '127.0.0.1',
  port: process.env.DB_PORT || 3306,
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || 'root',
  database: process.env.DB_NAME || 'Cityzen',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
```

```
module.exports = db;
```

Dockerfile αρχείο:

Το Dockerfile χρησιμοποιείται για τη δημιουργία Docker εικόνας για το backend της εφαρμογής Cityzen (Node.js app).

```
FROM node:18

WORKDIR /app

COPY package*.json ./

RUN npm install

# Install netcat-openbsd (for wait-for.sh script)

RUN apt-get update && apt-get install -y netcat-openbsd && rm -rf
/var/lib/apt/lists/*

COPY . .

COPY wait-for.sh /wait-for.sh

RUN chmod +x /wait-for.sh

CMD ["/wait-for.sh", "npm", "start"]
```

hashPassword.js αρχείο:

Το hashPassword.js είναι ένα script που χρησιμοποιεί τη βιβλιοθήκη bcryptjs για να μετατρέψει έναν απλό κωδικό (όπως "1234") σε έναν ασφαλή κρυπτογραφημένο κωδικό (hash).

```
// hashPassword.js
const bcrypt = require('bcryptjs');

const password = '1234';  // change this to the password you want to hash

bcrypt.hash(password, 10)
  .then(hash => {
    console.log('Hashed password:', hash);
  })
  .catch(err => {
    console.error('Error hashing password:', err);
  });
```

index.js αρχείο:

Το αρχείο index.js είναι το κύριο αρχείο εκκίνησης (entry point) του backend της εφαρμογής. Φορτώνει το Express app, διαμορφώνει τα middleware και τα routes, ελέγχει τη σύνδεση με τη βάση δεδομένων και ξεκινά τον server. Συγκεκριμένα,

- Φορτώνει τις ρυθμίσεις από το .env αρχείο
- Δημιουργεί και διαμορφώνει το Express app

- Συνδέεται με τη βάση δεδομένων
- Καθορίζει τις βασικές διαδρομές (routes) για το API
- Εκκινεί τον backend server στην πόρτα 5000

```
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const path = require('path');
const db = require('./db'); // ✅ Σύνδεση με τη βάση

const authRoutes = require('./routes/authRoutes');
const reportRoutes = require('./routes/reportRoutes');
const pointsRoutes = require('./routes/pointsRoutes');
const newsRoutes = require('./routes/newsRoutes');
const userreportsRoutes = require('./routes/userreportsRoutes');

// const citizenRoutes = require('./routes/citizenRoutes');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json());

// Serve static uploads (e.g. images)
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
```

```
// Routes
app.use('/api', authRoutes);
app.use('/api/reports', reportRoutes);
app.use('/api/points', pointsRoutes);
app.use('/api/news', newsRoutes);
app.use('/api/userreports', userreportsRoutes);
// app.use('/api/citizen', citizenRoutes);
// console.log('✅ /api/citizen route loaded');

// Root test routes
app.get('/', (req, res) => {
  console.log('✅ GET / route hit');
  res.send('City-Zen API is running! 🎉');
});

app.get('/test', (req, res) => {
  res.send('Test route works');
});

// ✅ Έλεγχος σύνδεσης με MySQL
db.query('SELECT 1')
  .then(() => console.log('✅ Connected to MySQL database!'))
  .catch(err => console.error('❌ Database connection failed:', err));

// Start server
app.listen(PORT, '0.0.0.0', () => {
  console.log(`🚀 Backend listening on http://localhost:${PORT}`);
```

});

FRONTEND

index.html αρχείο

Το index.html είναι το κύριο αρχείο HTML της frontend εφαρμογής μας. Είναι η βασική σελίδα που φορτώνει ο browser και μέσα της "ενσωματώνεται" το React app. Συγκεκριμένα,

- Είναι το σημείο εισόδου για την frontend εφαρμογή.

- Περιέχει το βασικό HTML skeleton (δομή) της σελίδας.

- Μέσα στο θα "φορτωθεί" το React app, το οποίο τοποθετεί δυναμικά το περιεχόμενο της εφαρμογής.

- Ο browser διαβάζει αυτό το αρχείο όταν ανοίγει την ιστοσελίδα και το React χρησιμοποιεί το root div για να εμφανίσει όλο το UI της εφαρμογής.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>City-Zen Citizen App</title>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

App.js αρχείο:

Το App.js είναι το κύριο αρχείο της React εφαρμογής μας και αναλαμβάνει τον καθορισμό της πλοήγησης (routing) και της προστασίας των σελίδων της εφαρμογής με βάση το αν ο χρήστης είναι συνδεδεμένος (μέσω token).

```
import React, { useState, useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import Login from './Login';
import HomePage from './HomePage';
import ReportPage from './ReportPage';
import ViewReportsPage from './ViewReportsPage';
import NewsPage from './NewsPage';
import PointsPage from './PointsPage';



function App() {
  const [token, setToken] = useState(localStorage.getItem('token'));

  // Optional: Keep token state in sync across tabs
  useEffect(() => {
    const onStorageChange = () => {
      setToken(localStorage.getItem('token'));
    };
    window.addEventListener('storage', onStorageChange);
    return () => window.removeEventListener('storage', onStorageChange);
  }, []);
```

```
// 🔒 Route guard component
const ProtectedRoute = ({ children }) => {
  return token ? children : <Navigate to="/login" replace />;
};


return (
  <Router>
    <Routes>
      <Route
        path="/login"
        element={
          token
            ? <Navigate to="/home" replace />
            : <Login setToken={setToken} />
        }
      />
      <Route path="/home" element={<ProtectedRoute><HomePage /></ProtectedRoute>} />
      <Route path="/report" element={<ProtectedRoute><ReportPage /></ProtectedRoute>} />
      <Route path="/view-reports" element={<ProtectedRoute><ViewReportsPage /></ProtectedRoute>} />
      <Route path="/news" element={<ProtectedRoute><NewsPage /></ProtectedRoute>} />
      <Route path="/points" element={<PointsPage />} />
      {/* Catch-all route */}
      <Route path="*" element={<Navigate to={token ? "/home" : "/login"} replace />} />
    </Routes>
```

```
    </Router>
  );
}

export default App;
```

index.js αρχείο:

ο index.js είναι αυτό που συνδέει τη React εφαρμογή με το HTML της σελίδας και ξεκινά την εμφάνιση της διεπαφής στον browser.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Pages:

Τα βασικά pages της εφαρμογής μας είναι τα εξής:

1) Login.js

```
import React, { useState } from 'react';
import axios from 'axios';
```

```
import { useNavigate } from 'react-router-dom';

function Login({ setToken }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');

    try {
      const response = await axios.post('http://localhost:5000/api/login', {
        username,
        password,
      });

      const token = response.data.token;

      if (token) {
        localStorage.setItem('token', token);
        setToken(token); // update parent state
        console.log("✅ Login successful, redirecting...");
        navigate('/home'); // redirect
      } else {
        setError("❌ Token not received");
      }
```

```
    } catch (err) {
      console.error(err);
      setError('Invalid username or password');
    }
  };

  return (
    <form onSubmit={handleSubmit} style={{ padding: '2rem', maxWidth: '300px', margin: 'auto' }}>
      <h1 style={{ textAlign: 'center', color: 'teal' }}>🌆 Welcome to City-Zen</h1>
      <h2>Login</h2>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <input
        type="text"
        placeholder="Username"
        value={username}
        onChange={e => setUsername(e.target.value)}
        required
        style={{ display: 'block', marginBottom: '1rem', width: '100%', padding: '0.5rem' }}
      />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        required
        style={{ display: 'block', marginBottom: '1rem', width: '100%', padding: '0.5rem' }}
```

```
      />
      <button type="submit" style={{ width: '100%', padding: '0.5rem' }}>Login</button>
    </form>
  );
}

export default Login;
```

2) HomePage.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';

const HomePage = () => {
  const [user, setUser] = useState(null);
  const [reports, setReports] = useState([]);
  const [userError, setUserError] = useState("");
  const [reportsError, setReportsError] = useState("");
  const [loadingUser, setLoadingUser] = useState(true);
  const [loadingReports, setLoadingReports] = useState(true);

  const token = localStorage.getItem('token');
  const navigate = useNavigate();

  useEffect(() => {
    if (!token) return;
```

```
setLoadingUser(true);
setLoadingReports(true);

axios.get('http://localhost:5000/api/users/me', {
  headers: { Authorization: `Bearer ${token}` },
})
  .then(res => {
    setUser(res.data);
    setUserError("");

    const userId = res.data.id;

    axios.get(`http://localhost:5000/api/userreports/citizen/${userId}`, {
      headers: { Authorization: `Bearer ${token}` },
    })
      .then(reportRes => {
        setReports(reportRes.data);
        setReportsError("");
      })
      .catch(() => setReportsError("❌ Failed to fetch user reports"))
      .finally(() => setLoadingReports(false));
  })
  .catch(() => {
    setUserError("❌ Failed to fetch user info");
    setLoadingUser(false);
    setLoadingReports(false);
  });
```

```
}, [token]);

const handleLogout = () => {
  localStorage.removeItem('token');
  window.location.href = "/login";
};

const handleReportClick = () => {
  navigate('/report');
};

const handleViewReportsClick = () => {
  navigate('/view-reports');
};

const handleNewsClick = () => {
  navigate('/news');
};

const handleGoToPoints = () => {
  navigate('/points');
};

return (
  <div style={{ padding: "2rem", fontFamily: "sans-serif" }}>
    <h1>Welcome {loadingUser ? "..." : user?.username || "Citizen"} 👋</h1>

    {/* 🎾 Button Section with Red Border */}
```

```
      <div style={{
        marginBottom: "1.5rem",
        padding: "1rem",
        border: "2px solid red",
        borderRadius: "10px",
        display: "flex",
        gap: "1rem",
        flexWrap: "wrap"
      }}>
        <button onClick={handleLogout} style={{ padding: "0.5rem 1rem", cursor: "pointer" }}>
          🔓 Logout
        </button>
        <button onClick={handleReportClick} style={{ padding: "0.5rem 1rem", cursor: "pointer" }}>
          📣 Report an Issue
        </button>
        <button onClick={handleViewReportsClick} style={{ padding: "0.5rem 1rem", cursor: "pointer" }}>
          🗂️ View All Reports
        </button>
        <button onClick={handleNewsClick} style={{ marginLeft: '1rem' }}>
          📰 Latest News
        </button>
        <button onClick={handleGoToPoints} style={{ marginLeft: '10px' }}>
          View My Points
        </button>
      </div>
```

```
      {userError && <p style={{ color: "red" }}>{userError}</p>}

      <h2>Your Reports</h2>
      {loadingReports ? (
        <p>Loading reports...</p>
      ) : reportsError ? (
        <p style={{ color: "red" }}>{reportsError}</p>
      ) : reports.length === 0 ? (
        <p>No reports found.</p>
      ) : (
        <ul>
          {reports.map(report => (
            <li key={report.report_id} style={{ marginBottom: "1rem", borderBottom: "1px solid #ccc", paddingBottom: "0.5rem" }}>
              <strong>Category:</strong> {report.category} <br />
              <strong>Description:</strong> {report.description} <br />
              {report.priority && <><strong>Priority:</strong> {report.priority} | </>}
              {report.status && <><strong>Status:</strong> {report.status} | </>}
              {report.citizen_name && <><strong>Citizen:</strong> {report.citizen_name} | </>}
              {report.location_address && <><strong>Location:</strong> {report.location_address}</>}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
};
```

```
export default HomePage;
```

3) ReportPage.js

```
import React, { useState } from 'react';
import axios from 'axios';

const ReportPage = () => {
  const [category, setCategory] = useState('');
  const [comment, setComment] = useState('');
  const [image, setImage] = useState(null);
  const [address, setAddress] = useState('');
  const [latitude, setLatitude] = useState('');
  const [longitude, setLongitude] = useState('');

  const token = localStorage.getItem('token');

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!category || !comment) {
      alert("Please fill out both category and comment.");
      return;
    }

    const formData = new FormData();
```

```
    formData.append('category', category);
    formData.append('comment', comment);
    if (image) formData.append('image', image);
    formData.append('address', address);
    formData.append('latitude', latitude);
    formData.append('longitude', longitude);

    try {
      const res = await axios.post('http://localhost:5000/api/reports', formData, {
        headers: {
          Authorization: `Bearer ${token}`,
          'Content-Type': 'multipart/form-data',
        },
      });
      alert('✅ Report submitted!');
    } catch (error) {
      console.error(error);
      alert('❌ Failed to submit report');
    }
  };

  return (
    <div style={{ padding: '2rem' }}>
      <h2>📣 Submit a Report</h2>
      <form onSubmit={handleSubmit}>
        <label>Category:</label><br />
        <select value={category} onChange={(e) => setCategory(e.target.value)}
required>
```

```
          <option value="">-- Select --</option>
          <option value="sidewalk">Sidewalk</option>
          <option value="lighting">Lighting</option>
          <option value="cleanliness">Cleanliness</option>
          <option value="other">Other</option>
        </select><br /><br />

        <label>Comment:</label><br />
        <textarea value={comment} onChange={(e) => setComment(e.target.value)}
required /><br /><br />

        <label>Image (optional):</label><br />
        <input type="file" onChange={(e) => setImage(e.target.files[0])} /><br /><br />

        <label>Address:</label><br />
        <input type="text" value={address} onChange={(e) => setAddress(e.target.value)}
/><br /><br />

        <label>Latitude:</label><br />
        <input type="text" value={latitude} onChange={(e) => setLatitude(e.target.value)}
/><br /><br />

        <label>Longitude:</label><br />
        <input type="text" value={longitude} onChange={(e) =>
setLongitude(e.target.value)} /><br /><br />

        <button type="submit">Submit Report</button>
      </form>
    </div>
```

```
  );
};

export default ReportPage;
```

4) PointsPage.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const PointsPage = () => {
  const [points, setPoints] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  const token = localStorage.getItem('token');

  useEffect(() => {
    const fetchPoints = async () => {
      if (!token) {
        setError('No token found. Please log in.');
        setLoading(false);
        return;
      }

      try {
        const res = await axios.get('http://localhost:5000/api/points/me', {
```

```
        headers: {
          Authorization: `Bearer ${token}`,
        },
      });

      // Defensive check in case the response structure changes
      if (res.data && typeof res.data.points === 'number') {
        setPoints(res.data.points);
      } else {
        setError('Unexpected response format.');
      }
    } catch (err) {
      console.error('Error fetching points:', err);
      setError('Failed to load points.');
    } finally {
      setLoading(false);
    }
  };

  fetchPoints();
}, [token]);

if (loading) return <p>Loading points...</p>;
if (error) return <p style={{ color: 'red' }}>{error}</p>;

return (
  <div style={{ padding: '1rem' }}>
    <h2>🎯 Your Points</h2>
```

```
      <p style={{ fontSize: '2rem', fontWeight: 'bold' }}>
        {points !== null ? `${points} pts` : 'No points found.'}
      </p>
    </div>
  );
};

export default PointsPage;
```

5) ViewReportsPage.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const ViewReportsPage = () => {
  const [reports, setReports] = useState([]);
  const [error, setError] = useState('');

  useEffect(() => {
    const fetchReports = async () => {
      try {
        const res = await axios.get('http://localhost:5000/api/reports', {
          headers: {
            Authorization: `Bearer ${localStorage.getItem('token')}`
          }
        });
```

```
        setReports(res.data);
      } catch (err) {
        console.error(err);
        setError('Failed to fetch reports');
      }
    };

    fetchReports();
  }, []);

  return (
    <div style={{ padding: '2rem' }}>
      <h2>All Reports</h2>
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <ul>
        {reports.map((r) => (
          <li key={r.id}>
            <strong>{r.category}</strong> - {r.description} | {r.status} |
{r.location_address}
          </li>
        ))}
      </ul>
    </div>
  );
};

export default ViewReportsPage;
```

6) NewsPage.js

```
import React, { useEffect, useState } from 'react';

const NewsPage = () => {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchPosts = async () => {
      try {
        const res = await fetch('http://localhost:5000/api/news/posts');
        const data = await res.json();
        setPosts(data);
      } catch (err) {
        console.error('Error fetching news:', err);
      } finally {
        setLoading(false);
      }
    };

    fetchPosts();
  }, []);

  if (loading) return <p>Loading news...</p>;

  return (
```

```
    <div style={{ padding: '2rem' }}>
      <h1>Latest News 📰</h1>
      {posts.length === 0 ? (
        <p>No announcements available.</p>
      ) : (
        posts.map(post => (
          <div key={post.id} style={{ marginBottom: '2rem' }}>
            <h2>{post.title}</h2>
            <p>{post.text}</p>
            {post.media_url && (
              <img
                src={post.media_url}
                alt="media"
                style={{ maxWidth: '100%', maxHeight: '300px' }}
              />
            )}
            <small>Posted on: {new Date(post.created_at).toLocaleString()}</small>
            <hr />
          </div>
        ))
      )}
    </div>
  );
};

export default NewsPage;
```

Dockerfile αρχείο:

Αυτό το Dockerfile δημιουργεί ένα development container για μια React εφαρμογή, ώστε να μπορείς να αναπτύσσεις τοπικά με live reload (hot reloading). Δεν κάνει build παραγωγής (npm run build), αλλά τρέχει την εφαρμογή με npm start για development.

```
# Development Dockerfile for React (no build stage, just npm start)

FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy only package files first to install dependencies
COPY package.json package-lock.json ./

# Install dependencies
RUN npm install

# Copy the rest of the app
COPY . .

# React dev server runs on 3000
EXPOSE 3000

# Ensure live reload works (Docker polling)
ENV CHOKIDAR_USEPOLLING=true
```

```
# Start React in dev mode
CMD ["npm", "start"]
```